使用 OpenAI Codex 開發病人時序圖應用：從 17 分鐘的實驗看 AI 開發模式的改變

摘要

近年來，AI 對軟體開發模式帶來了顯著改變。過去需要數天甚至數週才能完成的系統原型，如今可能在短時間內由 AI 協助完成。本文以一個實際案例為例：利用 **OpenAI Codex** 開發一個以 **FHIR 資料為基礎的病人時序圖（Patient Timeline）前端應用程式。**

在這次實驗中，透過 **GPT-5.3 Codex** 以及 **GPT-5.4** 兩種模型進行開發流程測試，從需求描述、系統設計、程式生成、測試到文件產出，整個過程僅花費約 **17 分 36 秒** 即完成可運作的 MVP 原型。

本文將介紹：

- Codex 協作開發流程
- 實際開發案例：FHIR 病人時序圖
- GPT-5.3 Codex 與 GPT-5.4 的實驗比較
- AI 開發與傳統開發模式的差異
- 未來軟體工程流程可能的轉變

透過這個案例，可以清楚看見 **AI Agent 型開發工具**正在重新定義軟體工程的效率與工作方式。

一、AI 協作式開發流程

在傳統軟體開發流程中，通常會經過需求分析、系統設計、開發、測試與文件等多個階段。而在 Codex 的協作模式中，整個流程可以被壓縮成一個 **AI 協作循環。**

在簡報中，整體開發流程可分為五個主要步驟：

1. 安裝與準備
2. 提出需求
3. 拆解執行工作
4. 驗證與修正
5. 產生文件

這五個步驟形成一個快速迭代的開發循環，使 AI 能夠逐步完成系統開發。

與傳統開發相比，最大的差異在於：

需求本身就可以直接變成開發指令。

換句話說，AI 不只是產生程式碼，而是開始扮演「協作開發者」的角色。

二、案例：FHIR 病人時序圖系統

這次實驗開發的系統是一個 **病人時序圖（Patient Timeline）前端應用程式。**

系統的資料來源是 **FHIR Server（Firely Server）**，並使用 Synthea 產生的模擬醫療資料。

FHIR 資料包含多種醫療資源，例如：

- Patient
- Encounter
- Condition
- Observation
- Procedure
- DiagnosticReport
- MedicationRequest
- DocumentReference
- Immunization
- Device

這些資料可以用來呈現病人的完整醫療歷程。

系統的主要功能包括：

1. 輸入 Patient ID
2. 從 FHIR Server 取得病人資料
3. 解析 Encounter 與相關醫療紀錄
4. 以時序圖呈現病人的就診與治療流程
5. 點擊事件查看詳細醫療資訊

換句話說，這個系統的核心目的，是將複雜的醫療資料轉換成**可視化的時間序列資訊**。

三、Codex 開發流程實驗

在這次實驗中，Codex 並不是直接一次產生完整系統，而是透過 **分階段任務（Steps）逐步完成開發**。

整個專案被拆解為 7 個主要工作：

1. 初始化專案
2. 建立 FHIR API 層
3. 建立資料轉換層
4. 建立頁面與 UI 元件
5. 加入狀態管理與快取
6. 撰寫測試
7. 產生 README 文件

這種方式其實非常接近 **現代軟體工程的模組化開發流程**。

Step 1：初始化專案

Codex 先建立一個前端專案，技術架構包括：

- React
- TypeScript
- Vite
- ESLint

- Prettier
- Vitest
- React Testing Library
- MUI
- TanStack Query
- Zustand

這一步耗時 **5 分 13 秒**。

也就是說，AI 已經能夠自動完成前端專案的初始化與工具配置。

Step 2：FHIR API 層

接著建立 API 存取層，例如：

- `getPatientById`
- `getEncountersByPatientAndDateRange`
- `getEncounterRelatedResources`
- `getPatientTimelineData`

這一步耗時 **2 分 12 秒**。

API 層負責從 FHIR Server 取得資料並進行整理。

Step 3：資料轉換層

FHIR 的 JSON 資料結構非常複雜，因此需要轉換為 UI 使用的 ViewModel，例如：

- PatientSummary
- TimelineEvent
- EncounterDetail

這一步耗時 **1 分 32 秒**。

Step 4：建立 UI 元件

系統主要 UI 元件包括：

- SearchPanel
- PatientSummaryCard
- TimelineView
- EncounterDetailDrawer

這一步耗時 **2 分 51 秒**。

UI 可以顯示：

- 病人摘要
- 就診時間軸

- Encounter 詳細資訊

Step 5：狀態管理與快取

為了提升效能與使用體驗，系統加入：

- **TanStack Query**：API 快取
- **Zustand**：UI 狀態管理

這一步耗時 **1 分 22 秒**。

Step 6：測試

Codex 自動加入：

- 單元測試
- UI 元件測試

測試項目包括：

- Mapper 正確性
- API 錯誤處理
- UI 互動行為

耗時 **2 分 3 秒**。

Step 7：文件產生

最後，AI 自動產生 README 文件，包括：

- 安裝方式
- 環境設定
- 啟動方法
- 測試方法

耗時 **49 秒**。

四、開發時間統計

整個專案完成時間：

17 分 36 秒

包含：

- 需求拆解：1 次
- 子任務：7 個
- 問題修正：1 次

這個結果顯示，AI 可以在非常短的時間內建立完整的 MVP 系統。

五、GPT-5.3 Codex vs GPT-5.4

在這次實驗中，也使用 **GPT-5.4** 再做一次測試。

與 GPT-5.3 Codex 的差異在於：

GPT-5.3 的方式是：

逐步完成任務

GPT-5.4 則嘗試：

一次生成整個系統

GPT-5.4 一次完成整個專案的時間約為：

11 分 1 秒。

最後生成的專案包含：

- React + TypeScript + Vite
- FHIR API
- Mapper
- UI 頁面
- README
- 測試

並且成功通過：

- npm lint
- npm test
- npm build

這代表 AI 已經可以**獨立完成一個完整 MVP 專案。**

六、AI 開發 vs 傳統開發

如果使用傳統方式開發同一個系統，流程通常會是：

1. 系統需求分析
2. 架構設計
3. 專案初始化
4. API 開發
5. UI 開發
6. 測試
7. 文件

即使是一個簡單的 MVP，也可能需要：

2 天到 1 週

而 AI 協作開發模式的特徵包括：

1. 需求即程式

自然語言需求可以直接變成程式碼。

---

2. AI 自動拆解任務

AI 會把系統拆成多個可執行步驟。

---

3. 自動生成文件

README、測試、架構都能自動產生。

---

4. 快速原型開發

可以在短時間內建立 MVP。

七、AI 開發模式的意義

這次實驗並不是在證明 AI 可以完全取代工程師。

更重要的是：

軟體開發的角色正在改變。

未來工程師的角色可能會從：

寫程式的人

變成：

設計系統與指揮 AI 的人

換句話說：

Prompt 可能會變成新的「程式語言」。

八、結論

這次利用 OpenAI Codex 開發病人時序圖系統的實驗顯示，AI 已經可以在極短時間內建立一個可運作的應用程式原型。整個專案從需求描述到程式完成僅花費約 **17 分 36 秒**，而使用 GPT-5.4 甚至能在 **11 分鐘左右**完成整個專案。

這意味著軟體開發的模式正在快速改變。過去需要大量時間撰寫程式碼與配置環境的工作，未來可能只需要清楚描述需求與設計架構，AI 即可協助完成大部分實作。

然而，AI 開發並不代表工程師變得不重要。相反地，工程師的角色將更加偏向：

- 系統設計
- 架構規劃
- 問題拆解
- AI 協作與驗證

AI 將成為工程師最強大的開發助手，而不是替代者。

對於軟體產業而言，這或許代表著一個新的時代：

從「人工寫程式」邁向「AI 協作開發」。